

Graphs Algorithms

1. Topological Sort (Kahn's algorithm)

- $O(V+E)$
- Only valid for DAG.
- Used for course schedule, dependencies problems.
- Edge Case: Can detect cycles if $\text{len}(\text{topo_order}) \neq N$
- Steps:
 - Compute indegree for all vertices.
 - Push all nodes with indegree 0 into queue.
 - Pop node, add to order, decrease indegree of neighbors.
 - If indegree becomes 0, push to queue.
 - Detect cycle if output length $\neq$ number of nodes.

```
## Input:      N, edges
## Output:     Sorted Order
```

```
G = defaultdict(list)
indegree = defaultdict(int)
for u, v in edges:
    G[u].append(v)
    indegree[v] += 1

q = deque()
for i in range(N):
    if indegree[i] == 0:
        q.append(i)

topo_order = []
while q:
    u = q.popleft()
    topo_order.append(u)
    for v in G[u]:
        indegree[v] -= 1
        if indegree[v] == 0:
            q.append(v)

print(topo_order)
```

2. Dijkstra's Algorithm (single source shortest path)

- $O(V \log E)$
- Works for non-negative weighted graphs
- Used to find the shortest path from node u to all other nodes.
- Steps:
 - Initialize $dist[]$ with infinity, $source=0$.
 - Use min-heap (priority queue) to pick min distance node.
 - Relax all edges of picked node.
 - Update distance and push neighbor if relaxed.

Input: N , edges, start_node

Output: $dist$: Shortest distance of each node from u

```
G = defaultdict(list)
```

```
for u, v, w in edges:
```

```
    G[u].append((w, v))
```

```
dist = [float('inf')] * N
```

```
dist[start_node] = 0
```

```
pq = [(0, start_node)] # distance, node
```

```
while pq:
```

```
    d, u = heappop(pq)
```

```
    if d > dist[u]:
```

```
        continue
```

```
    for w, v in G[u]:
```

```
        if d+w < dist[v]:
```

```
            dist[v] = d+w
```

```
            heappush(pq, (d+w, v))
```

```
return dist
```

3. Bellman-Ford (single source, negative weights)

- $O(V \cdot E)$
- Works for negative weights, detects negative weight cycles.
- Steps:
 - Initialize `dist[]` with infinity, `source=0`.
 - Relax all edges $V-1$ times.
 - Check for negative weight cycles on one extra iteration.

Input: `V, edges, src`

Output: `dist`: distance of each node from `src`

`MAX = 10**8`

`dist = [MAX] * V`

`dist[src] = 0`

for `i` in `range(V-1)`:

 for `u, v, w` in `edges`:

 if `dist[u] != MAX` and `dist[u]+w < dist[v]`:

`dist[v] = dist[u] + w`

for `u, v, w` in `edges`:

 if `dist[u] != MAX` and `dist[u]+w < dist[v]`:

 return `[-1]`

return `dist`

4. Floyd Warshall (multi source shortest path)

- $O(V^3)$
- Works for negative edges but no negative cycles
- All pairs shortest path
- Steps:
 - Initialize matrix $\text{dist}[i][j]$.
 - For each k , update $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.
 - Detect negative cycles by checking $\text{dist}[i][i] < 0$.

Input: dist: Adjacency Matrix

Output: dist: Updated to shortest path

$V = \text{len}(\text{dist})$

```
for k in range(V):
    for i in range(V):
        for j in range(V):
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]
```

detect negative cycles

```
for i in range(V):
    if dist[i][i] < 0:
        print("Negative cycle detected")
```

return dist

5. Prim's Algorithm (MST)

- $O(E \log V)$
- Connected Weighted Graphs
- Steps:
 - Start from any node, mark as visited.
 - Use priority queue to pick smallest edge.
 - Add node to MST and repeat until all nodes are visited.

Input: V, edges

Output: mst_cost

```
G = defaultdict(list)
for u, v, w in edges:
    G[u].append((w, v))
    G[v].append((w, u))

vis = [0]*V
pq = [(0, 0, -1)]
res = 0
while pq:
    dist, node, parent = heappop(pq)
    if vis[node]:
        continue
    vis[node] = 1
    res += dist
    for w, neigh in G[node]:
        if not vis[neigh]:
            heappush(pq, (w, neigh, node))

return res
```

6. Kruskal's Algorithm (MST, using DSU)

- $O(E \log E)$
- Graphs should be connected
- Used in edge heavy graphs
- Steps:
 - Sort all edges by weight.
 - Initialize DSU structure.
 - For each edge, if nodes are not in same set, take edge and union sets.
 - Stop when $V-1$ edges taken.

Input: V , edges

Output: mst_cost

```
G = defaultdict(list)
    for u, v, w in edges:
        G[u].append((w, v))
        G[v].append((w, u))

ds = DSU(V)
edges.sort(key=lambda x:x[2])
res = 0
for u, v, w in edges:
    if ds.find(u) == ds.find(v):
        continue
    res += w
    ds.unionByRank(u, v)
return res
```

7. DSU

- $O(4 \alpha)$ -> almost constant
- Steps:
 - Initialize parent[] and rank[]/size[].
 - Path compression for find().
 - Union by rank or size.

class DSU:

```
def __init__(self, N):
    self.parent = list(range(N))
    self.size = [1] * N
    self.rank = [1] * N
```

```
def find(self, x):
    if self.parent[x] != x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]
```

```
def unionBySize(self, x, y):
    root_x = self.find(x)
    root_y = self.find(y)
    if self.parent[root_x] == self.parent[root_y]:
        return
    if self.size[root_x] > self.size[root_y]:
        self.parent[root_y] = root_x
        self.size[root_x] += self.size[root_y]
    else:
        self.parent[root_x] = root_y
        self.size[root_y] += self.size[root_x]
    return
```

```
def unionByRank(self, x, y):
    root_x = self.find(x)
    root_y = self.find(y)
    if self.parent[root_x] == self.parent[root_y]:
        return
    if self.rank[root_x] == self.rank[root_y]:
        self.parent[root_x] = root_y
        self.rank[root_x] += 1
    elif self.rank[root_x] > self.rank[root_y]:
        self.parent[root_y] = root_x
    else:
        self.parent[root_x] = root_y
    return
```

8. Bridges in Graph (Tarjan's Algorithm)

- $O(V+E)$
- Graph can be directed/undirected.
- Steps:
 - DFS traversal, maintain discovery & low times.
 - If $low[child] > disc[node]$, edge is a bridge.

Input: V, G:adj_list

Output: bridges: list of edges which form a bridge

```
def dfs(node, parent):
    nonlocal timer
    visited[node] = 1
    time[node] = low[node] = timer
    timer += 1
    for neigh in G[node]:
        if neigh == parent: continue
        if not visited[neigh]:
            dfs(neigh, node)
            low[node] = min(low[node], low[neigh])
            if low[neigh] > time[node]:
                bridges.append([node, neigh])
        else:
            low[node] = min(low[node], low[neigh])

bridges = []
timer = 0
time = [0]*N
low = [0]*N
visited = [0]*N

dfs(0, -1)
print(bridges)
```


9. Articulation Points

- $O(V+E)$
- Steps:
 - DFS traversal with discovery and low times.
 - Root with more than 1 child is articulation.
 - For others, if $low[child] \geq disc[node]$.

Input: V, G:adj_list

Output: articulation_points

```
def dfs(node, parent):
    nonlocal timer
    visited[node] = 1
    time[node] = low[node] = timer
    timer += 1
    count = 0
    for neigh in G[node]:
        if neigh == parent: continue
        if not visited[neigh]:
            dfs(neigh, node)
            low[node] = min(low[node], low[neigh])
            if low[neigh] >= time[node] and parent != -1:
                mark[node] = 1
        else:
            low[node] = min(low[node], time[neigh])

    if parent == -1 and count > 1:
        mark[node] = 1

bridges = []
timer = 0
time = [0]*N
low = [0]*N
visited = [0]*N

for i in range(N):
    if not visited[i]:
        dfs(i, -1)

articulation_points = [i for i in range(N) if mark[i]]
print(articulation_points)
```

10. Kosaraju's Algorithm (strongly connected components)

- $O(V+E)$
- For directed Graphs
- Steps:
 - Perform DFS and push finish order to stack.
 - Reverse graph.
 - Perform DFS on reversed graph in stack order.

Input: adj_list

Output: scc: number of strongly connected components

class Solution:

```
def kosaraju(self, adj):
```

```
    def dfs(node):
```

```
        vis[node] = 1
```

```
        for neigh in adj[node]:
```

```
            if not vis[neigh]:
```

```
                dfs(neigh)
```

```
        stack.append(node)
```

```
    def dfs2(node):
```

```
        vis[node] = 1
```

```
        for neigh in inv[node]:
```

```
            if not vis[neigh]:
```

```
                dfs2(neigh)
```

```
    N = len(adj)
```

```
    vis = [0]*N
```

```
    stack = []
```

```
    for i in range(N):
```

```
        if not vis[i]:
```

```
            dfs(i)
```

```
    inv = [[] for i in range(N)]
```

```
    for i in range(N):
```

```
        vis[i] = 0
```

```
        for neigh in adj[i]:
```

```
            inv[neigh].append(i)
```

```
    scc = 0
```

```
    for i in range(N):
```

```
        node = stack.pop()
```

```
        if not vis[node]:
```

```
            dfs2(node)
```

```
            scc += 1
```

```
    return scc
```